JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 : 202212941 이름: 박준호 제출일: 2026-10-28

서비스 이름 : 시네로그 – 영화 리뷰 게시판

기술 조합과 버전 : React 19.3.0 (Vite 8.3.4) / Node.js Express 5.3.0 (jsonwebtoken 9.0.3, sql.js 1.14.2) / Node.js v24.13.0, Windows 11 Home, Chrome

GitHub 저장소 URL : https://github.com/JH9654/JWT-Assignment

소스 코드 기준 : main 브랜치 / 코드 구현 기준 커밋

[서비스 소개와 선택 이유 작성]

시네로그는 영화를 본 뒤 감상을 기록하고 다른 사람과 나누고 싶은 사용자를 위한 영화 리뷰 게시판이다. 로그인하지 않은 사람도 리뷰 목록과 상세 내용을 읽을 수 있지만, 리뷰를 쓰려면 로그인해야 하고 수정과 삭제는 리뷰를 쓴 본인만 할 수 있다. 로그인은 JWT 로 처리하며, 누가 요청했는지와 그 사람이 글의 작성자인지는 서버가 토큰을 검증해서 판단한다.

기술 조합은 React 와 Express 로 정했다. 프론트와 백엔드가 모두 JavaScript 라서 화면 이벤트부터 api.js, 백엔드 라우트, 인증 함수까지 이어지는 요청 흐름을 한 언어로 따라갈 수 있다고 판단했다. 제공 템플릿에서는 인증 관련 TODO(backend-express/src/auth.js, frontend-react/src/api.js)를 구현하고, main.jsx 의 오류 처리와 화면 안내를 보완했으며, 화면 문구를 영화 리뷰 주제에 맞게 바꿨다. API 명세와 데이터 구조(title, content), SQLite 경로(data/board.sqlite)는 바꾸지 않았다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	App.js(제공), main.jsx	토큰 없이 200
로그인과 JWT 발급	완료	Auth.js loadConfig-issueToken / api.js login	HS256, sub-iss-aud-exp, TTL 기본 90 초
인증과 내 정보 조회	완료	Auth.js currentUserId / app.js / api/me	토큰 없음,변조,만료 401 구분
본인 글 작성·수정·삭제	완료	Auth.js asserOwner / api.js canEdit	타인 변경 403, 작성자 위조 400
만료·로그아웃·오류 표시	완료	Api.js request / main.jsx run	401 이면 로그인 해제, 403 이면 유지
DB 데이터 유지	완료	Store.js(제공), data/board.sqlite	서버 재시작 후 유지

구조와 변경 내역

브라우저는 http://127.0.0.1:4173 (vite preview)에서 빌드된 화면을 받는다. 화면의 모든 API 요청은 /api 로 보내지고, vite.config.js 의 preview.proxy 가 이를 http://127.0.0.1:8000 의 Express 서버로 전달한다. Express 는 SQLite 파일(backend-express/data/board.sqlite)에 게시글을 저장한다.

[브라우저: React (4173)] - /api 프록시 -> [Express (8000)] -> [SQLite data/board.sqlite]

(1) 로그인 흐름

로그인 폼 -> api.js login() -> POST /api/auth/login -> app.js: fields() 입력 검사 -> verifyCredentials() 비밀번호 확인 -> auth.js issueToken() 으로 JWT 발급 -> {accessToken, expiresIn, user}

<- api.js 가 accessToken.user 를 메모리 변수 session 에만 저장 (localStorage 사용 안 함)

(2) 게시글 수정 흐름

수정 저장 -> api.js request()가 Authorization: Bearer <JWT> 첨부 -> PUT /api/posts/{id}

- ➔ Auth.js currentUserId() : JWT 검증 -> 사용자 ID (실패 시 401)
- ➔ App.js getPost() : 글 존재 확인 (없으면 404)
- ➔ Auth.js assertOwner() : 작성자 비교 (다르면 403)
- ➔ App.js fields() : title,content 검사 (오류 시 400)
- ➔ Store.update() : DB 변경 -> 200

토큰은 로그인 응답으로 받아 브라우저 메모리(api.js 의 session 변수)에만 보관한다. /api/me 와 글 등록, 수정, 삭제 요청에만 Authorization 헤더로 보내고, 공개 목록, 상제 조회는 토큰이 없어도 된다.

새로고침하면 메모리가 초기화되어 로그아웃 상태가 된다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-express/src/auth.js loadConfig(), readExampleSecret() (추가	없음	서버 시작 시 JWT_SECRET 이 비었는지, 64 자리 16 진수인지, 예시 키인지 검사하고 32 바이트 키로 변환. TOKEN_TTL 검사	AI 초안 → 본인 실행 검증
auth.js issueToken()	501 NOT_IMPLEMENTED	HS256, sub(문자열 ID)·iss·aud·exp 를 넣어 JWT 발급	AI 초안 → 본인 실행 검증

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
auth.js currentUserId()	데모 ID 1 반환	Bearer 추출, jwt.verify 검증, exp·sub 형식·실제 계정 확인, 오류 code 구분	AI 초안 → 본인 실행 검증
auth.js assertOwner()	검사 없음	검증된 사용자 ID 와 글의 authorId 가 다르면 403 FORBIDDEN	AI 초안 → 본인 실행 검증
frontend-react/src/api.js login, authorizationHeaders, request, canEdit	TODO 상태	토큰을 메모리에 저장, Bearer 헤더 첨부, 401 이면 세션 해제, code 별 안내 문구, 작성자일 때만 수정 버튼 표시	AI 초안 → 본인 실행 검증
frontend-react/src/main.jsx run(), 배너, 로그아웃, 작성 카드	기본 화면	404 이면 선택 해제와 목록 갱신, 데모 배너를 서비스 안내로 교체, 로그아웃 시 폼 초기화, 비로그인 시 작성 안내와 등록 버튼 비활성화, 영화 리뷰 문구	AI 초안 → 본인 반영
frontend-react/index.html	탭 제목	"시네로그 | 영화 리뷰 게시판"	본인 주제 결정
backend-express/.env (커밋 안 함)	-	직접 생성한 JWT_SECRET 설정	본인
README.md	과제 안내문	제출용 README 로 교체	AI 초안 → 본인 반영

로컬 실행 기록

실제로 사용한 설치·빌드·실행 명령은 아래와 같다. 자세한 재현 방법은 저장소의 README.md(https://github.com/JH9654/JWT-Assignment)에 정리했다. 명령은 VS Code 터미널(PowerShell)에서 실행했다.

터미널 A – 백엔드 (backend-express 폴더)

Npm ci

Copy-Item .env.example .env

Npm start

{:status":"ok", "stack":"express"}

터미널 B – 프론트 (frontend-react 폴더)

Npm ci

Npm run build

Npm run preview

만료 테스트 시 (터미널 A)

$env:TOKEN_TTL="3"; npm start

운영체제와 버전 : Windows 11 Home / Node.js v24.13.0 / Python 3.14 / Chrome

환경설정 : backend-express/.env — JWT_SECRET(직접 생성, 커밋하지 않음), JWT_ISSUER=webservice-class, JWT_AUDIENCE=board-api, TOKEN_TTL=90

SQLite 파일 경로 : backend-express/data/board.sqlite (서버 첫 실행 시 자동 생성, 커밋하지 않음)

프론트 빌드 : npm run build 후 npm run preview 로 4173 번 실행

4173 번 접속과 API 연결 : 확인 — 빌드 화면에서 보낸 요청의 Request URL 이 http://127.0.0.1:4173/api/...이고 200 응답 (T01, T02)

서버 재시작 후 데이터 유지 : 확인 — 백엔드를 Ctrl+C 로 종료 후 같은 폴더에서 npm start, "재시작 테스트" 글·작성자·내용 유지 (T15)

증거 위치 : docs/evidence/T01_public.png, T02_login_status.png, T15_before.png, T15_after.png

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수 : backend-express/src/app.js POST /api/auth/login (제공), backend-express/src/auth.js loadConfig(), issueToken()

코드 발췌

App.js (제공 코드)

```
const input = fields(req.body, { email: 254, password: 128 });

const user = verifyCredentials(input.email, input.password);

if (!user) throw new ApiError(401, "INVALID_CREDENTIALS", ...);

const accessToken = issueToken(user);

res.json({ accessToken, expiresIn: Number(process.env.TOKEN_TTL || 90), user });


// auth.js

export function issueToken(user) {

  return jwt.sign({}, config.key, {

    algorithm: "HS256", subject: String(user.id),

    issuer: config.issuer, audience: config.audience, expiresIn: config.ttl,

  });

}
```

동작과 응답 코드 설명

입력: JSON 본문의 email, password.

처리: fields()가 허용된 필드와 길이를 검사하고, 제공된 verifyCredentials()가 scrypt 해시로 비밀번호를 비교한다. 일치하면 issueToken()이 jsonwebtoken 라이브러리로 HS256 서명 JWT 를 만든다. 토큰에는 sub(사용자 ID 문자열), iss(webservice-class), aud(board-api), exp(발급 시각 + TOKEN_TTL 초)만 들어가고, 비밀번호나 해시는 넣지 않는다. 서명 키는 서버가 시작할 때 loadConfig()가 .env 의 JWT_SECRET 을 읽어, 비어 있거나 64 자리 16 진수가 아니거나 .env.example 의 예시 키와 같으면 서버 시작을 거부한 뒤 Buffer.from(hex)로 32 바이트 키로 바꾼 것이다.

출력: 200 과 { accessToken, expiresIn, user{id,name,email} } (T02).

실패: 틀린 비밀번호와 없는 이메일은 똑같이 401 INVALID_CREDENTIALS 로 응답해 어떤 이메일이 존재하는지 드러나지 않게 한다 (T03). 형식이 틀리면 400 BAD_INPUT 이다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수 : frontend-react/src/api.js login(), authorizationHeaders(), request(), clearSession()

코드 발췌

```
let session = { accessToken: "", user: null };


function authorizationHeaders() {
  return session.accessToken ? { Authorization: `Bearer ${session.accessToken}` } : {};
}
export async function login(email, password) {
  const data = await request("/auth/login", { method: "POST", body: JSON.stringify({ email, password }) });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}
// request() 안
if (!response.ok) {
  if (response.status === 401) clearSession();
  ...
}
```

동작과 응답 코드 설명

입력: 로그인 폼의 email, password 와 서버의 로그인 응답.

처리: login()은 응답의 accessToken 과 user 를 모듈 변수 session 에 저장한다. localStorage 나 sessionStorage 에는 저장하지 않는다. 모든 API 요청이 거치는 request()는 authorizationHeaders()로 토큰이 있을 때만 Authorization 헤더를 붙인다. 그래서 토큰이 없을 때 "Bearer undefined" 같은 헤더가 만들어지지 않고, 공개 조회는 헤더 없이 나간다 (T01). 204 응답은 JSON 으로 파싱하지 않는다.

출력: 로그인하면 화면에 "Alice 님 로그인 중"이 표시되고, /me 와 글 등록·수정·삭제 요청에 Bearer 헤더가 실제로 붙는 것을 Network 에서 확인했다 (T05).

실패와 로그아웃: 401 을 받으면 clearSession()으로 토큰과 사용자를 지우고 code 에 맞는 안내(로그인 필요, 유효하지 않음, 만료)를 보여준다. 403 은 토큰이 유효한 상태이므로 로그인을 유지한다. 로그아웃 버튼은 clearSession()과 화면 상태 초기화만 한다. 이미 복사된 토큰을 서버에서 무효화하지는 않으며, 그 토큰은 exp 가 지나야 사용할 수 없게 된다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수 : backend-express/src/auth.js currentUserId()

코드 발췌

```
const match = /^Bearer (\S+)$/.exec(req.get("Authorization") ?? "");
if (!match) throw new ApiError(401, "TOKEN_REQUIRED", ...);
try {
  payload = jwt.verify(match[1], config.key, {
    algorithms: ["HS256"], issuer: config.issuer, audience: config.audience,
  });
} catch (error) {
  if (error.name === "TokenExpiredError") throw new ApiError(401, "TOKEN_EXPIRED", ...);
  throw new ApiError(401, "INVALID_TOKEN", ...);
}
if (typeof payload.exp !== "number") throw new ApiError(401, "INVALID_TOKEN", ...);
if (typeof payload.sub !== "string" || !/^[1-9]\d*$/.test(payload.sub)) throw ...INVALID_TOKEN;
const userId = Number(payload.sub);
if (!findPublic(userId)) throw new ApiError(401, "INVALID_TOKEN", ...);
return userId;
```

동작과 응답 코드 설명

입력: 요청의 Authorization 헤더.

처리: 먼저 "Bearer <토큰>" 형식인지 확인한다. 그다음 jwt.decode()가 아니라 jwt.verify()로 검증한다. decode 는 내용을 읽기만 하므로 위조된 토큰도 그대로 읽히지만, verify 는 서명, 허용 알고리즘(HS256 만), 만료, iss, aud 를 모두 확인한다. jsonwebtoken 은 exp 가 아예 없는 토큰을 만료 검사 없이 통과시키기 때문에 exp 가 있는지 직접 한 번 더 확인하고, sub 가 양의 정수 문자열인지와 실제 계정이 있는지도 확인한다. 여기까지 통과한 사용자 ID 를 함수의 반환값으로 /me 와 글 등록·수정·삭제 처리에 넘기는 부분이 이 구현의 요청 주체 연결점이다. 요청 본문이나 토큰에 적힌 사용자 정보를 검증 없이 쓰지 않는다.

출력: 검증된 사용자 ID(숫자). /api/me 는 이 ID 로 공개 정보만 반환한다 (200, T05).

실패: 헤더 없음·형식 오류 → 401 TOKEN_REQUIRED (T04, T11), 서명 변조·클레임 오류·없는 사용자 → 401 INVALID_TOKEN (T09), 정상 서명의 만료 토큰 → 401 TOKEN_EXPIRED (T10). 401 응답에는 app.js 의 공통 오류 처리기가 WWW-Authenticate: Bearer 헤더를 붙인다.

3 4 작성자 권한 검사

파일 / 함수 : backend-express/src/auth.js assertOwner(), app.js PUT·DELETE /api/posts/:id (제공)

코드 발췌

```
// app.js (제공): 인증 → 글 존재 → 권한 → 입력 검증 → DB 변경
const userId = currentUserId(req), id = postId(req), p = getPost(id);
assertOwner(userId, p.authorId);
res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));
// auth.js
export function assertOwner(userId, authorId) {
 if (userId !== Number(authorId))
  throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다.");
}
```

동작과 응답 코드 설명

비교 대상: currentUserId()가 토큰을 검증해서 돌려준 사용자 ID 와, DB 에 저장된 글의 authorId 를 비교한다. 이 비교가 권한 경계 대조이며, 둘이 다르면 403 FORBIDDEN 을 던져 store.update()나 store.remove()까지 가지 않는다. 그래서 거부된 요청 뒤에 글을 다시 조회하면 updatedAt 까지 그대로다 (T07, T08).

401 과 403 의 차이: 401 은 누가 요청했는지 확인하지 못한 경우(토큰 없음·변조·만료)이고, 403 은 사용자가 확인됐지만 그 글의 작성자가 아닌 경우다. 그래서 프론트는 401 이면 로그아웃시키고 403 이면 로그인을 유지한다. 처리 순서 때문에 Bob 이 Alice 글에 공백 제목을 보내면 입력 오류(400)보다 권한 오류(403)가 먼저 나온다 (판단 순서 확인).

작성자 위조 방지: 글을 등록할 때 작성자는 요청 본문이 아니라 currentUserId()의 결과로 서버가 정한다 (store.create(..., userId)). 본문에 authorId 나 userId 를 넣으면 fields()가 400 BAD_INPUT 으로 거부한다 (T12). 프론트의 canEdit()가 다른 사람 글의 버튼을 숨기지만 이것은 화면 편의일 뿐이다. 버튼을 거치지 않고 Bob 의 토큰으로 직접 요청해도 서버가 403 으로 막는다.

테스트 결과

실행 환경: Windows 11 Home / Node.js v24.13.0 / Python 3.14 / Chrome / VS Code 터미널(PowerShell)

로컬 배포 확인: 4173 번 빌드 화면에서 로그인·조회 요청의 Request URL 이 http://127.0.0.1:4173/api/...이고 200 으로 응답함을 확인했다 (T01, T02)

테스트 일시 및 코드 버전: 2026-10-27 ~ 2026-10-28 / 코드 버전

토큰 TTL: 일반 테스트 90 초 / 만료 테스트 3 초 ($env:TOKEN_TTL="3" 후 서버 재시작)

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인(Authorization 없음) 상태에서 글 상세 GET /api/posts/7, 목록 조회	200 공개 조회	200 / 해당 없음, Request headers 에 Authorization 없음	PASS	T01_public.png, T16_refresh.png
T02	Alice 로그인 POST /api/auth/login	200, JWT·user, 화면 표시	200 / 해당 없음, accessToken(가림)·expiresIn 90·user{id 1, Alice}, "Alice 님 로그인 중"	PASS	T02_login_status.png, T02_login_response.png
T03	틀린 비밀번호(alice) / 없는 이메일로 각각 로그인	401 INVALID_CREDENTIALS, 토큰 미발급	401 / 401, 화면 "이메일 또는 비밀번호가 올바르지 않습니다", 로그아웃 상태 유지	PASS	T03_wrong_pw.png, T03_unknown_email.png
T04	토큰 없이 GET /api/me, POST /api/posts	각각 401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED (WWW-Authenticate: Bearer), 401 / TOKEN_REQUIRED	PASS	T04_no_token.png
T05	Alice 토큰으로 리뷰 등록 후 /me 조회	201, authorId 1, /me Alice	201 / 해당 없음 (Authorization: Bearer 첨부), /me 200 (※ Location·authorId 는 이 캡처에 없음. Location 은 T17, authorId 1 은 T07 조회 결과에서 확인)	PASS	T05_create.png, T05_me.png
T06	Alice 가 본인 글(9 번) 수정·삭제	200, 204, DB 반영	PUT 200, DELETE 204 No Content, 목록 2 개 → 1 개	PASS	T06_put.png, T06_delete_204.png
T07	Bob 토큰으로 Alice 글(7 번) PUT	403 FORBIDDEN, 원문 유지	403 / FORBIDDEN, 재조회 시 title·updatedAt 동일	PASS	T07_T08_bob_forbidden.png
T08	Bob 토큰으로 Alice 글(7 번) DELETE	403 FORBIDDEN, 글 유지	403 / FORBIDDEN, 재조회 200 글 유지	PASS	T07_T08_bob_forbidden.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T09	Alice 토큰 서명부 첫 글자 변조 후 GET /me, POST, PUT, DELETE	401 INVALID_TOKEN, 변경 없음	4 건 모두 401 / INVALID_TOKEN (WWW-Authenticate: Bearer), 재조회 200 원문 유지	PASS	T09_tampered.png
T10	TTL 3 초 발급 후 5 초 대기, 화면 /me 조회와 PUT·DELETE	401 TOKEN_EXPIRED, 로그인 해제·안내	화면: /me 401 / TOKEN_EXPIRED, 만료 안내, "로그아웃 상태". API: PUT·DELETE 401 / TOKEN_EXPIRED, 재조회 200 유지	PASS	T10_expired_ui.png, T10_expired_api.png
T11	Alice 로그인 후 로그아웃, 내 정보 조회	헤더 없음, 401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED, "로그인이 필요합니다", "로그아웃 상태"	PASS	T11_logout_me.png
T12	Bob 토큰으로 등록 요청에 authorId:1 / userId:1 추가	400 BAD_INPUT, 위조 글 미생성	400 / BAD_INPUT 2 건, 목록 재조회 시 위조 글 없음	PASS	T12_forge_author.png
T13	Alice 토큰으로 공백 제목, 공백 본문, 제목 101 자, 내용 5001 자	400 BAD_INPUT	4 건 모두 400 / BAD_INPUT	PASS	T13_validation.png
T14	없는 ID 99999 의 조회·수정·삭제(정상 토큰), GET /api/posts/abc	404, 400	404 / NOT_FOUND 3 건, 400 / BAD_INPUT	PASS	T14_not_found.png
T15	Alice 로 "재시작 테스트" 작성 → 백엔드 Ctrl+C 후 npm start → 새로고침	글·작성자·내용 유지	목록 2 개에 "재시작 테스트"(Alice) 유지, GET /api/posts/12 200, 상세 내용 유지	PASS	T15_before.png, T15_after.png
T16	Alice 로그인 후 F5 새로고침	공개 글 유지, 로그인 해제	"로그아웃 상태", GET /api/posts 200 목록 유지	PASS	T16_refresh.png
T17	토큰 없이 PUT·DELETE 7 번, Bob 글 작성(11 번)·본인 수정·삭제	무토큰 401, Bob 본인 작업 성공	401 / TOKEN_REQUIRED 2 건, 201 (Location: /api/posts/11, authorId 2), 200 (authorId·createdAt 유지), 204	PASS	T17_no_token_and_bob_own.png

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200 (상세 조회, 로그인, /me, 본인 수정)	T01, T02, T05, T06
새 글 등록	**201 및 Location**	201, Location: /api/posts/11	T05, T17
본인 글 삭제	**204 및 빈 본문**	204 No Content, 본문 없음	T06, T17
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT	T12, T13, T14
잘못된 계정	**401 INVALID_CREDENTIALS**	401 (화면 안내로 code 확인)	T03
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED	T04, T11, T17
변조·클레임 오류	**401 INVALID_TOKEN**	401 / INVALID_TOKEN	T09
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED	T10
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN	T07, T08, 판단 순서
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND	T14, 판단 순서

WWW-Authenticate: 토큰 없는 /api/me(T04)와 변조 토큰 /api/me(T09)의 401 응답 헤더에 WWW-Authenticate: Bearer 가 있음을 curl -i 로 확인했다.

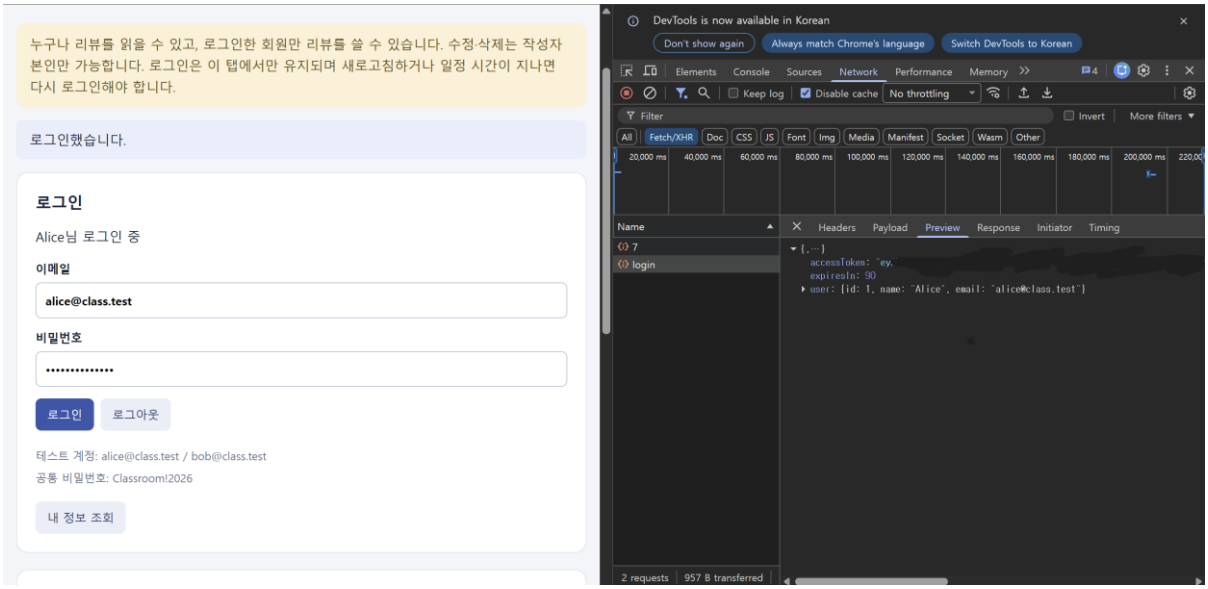
401 과 403 의 화면 차이: 401(T10, T11)을 받으면 화면이 즉시 "로그아웃 상태"로 바뀌고 만료·로그인 필요 안내가 표시된다. 403 은 토큰이 유효한 상태라서 api.js 가 세션을 지우지 않고 권한 안내만 표시한다.

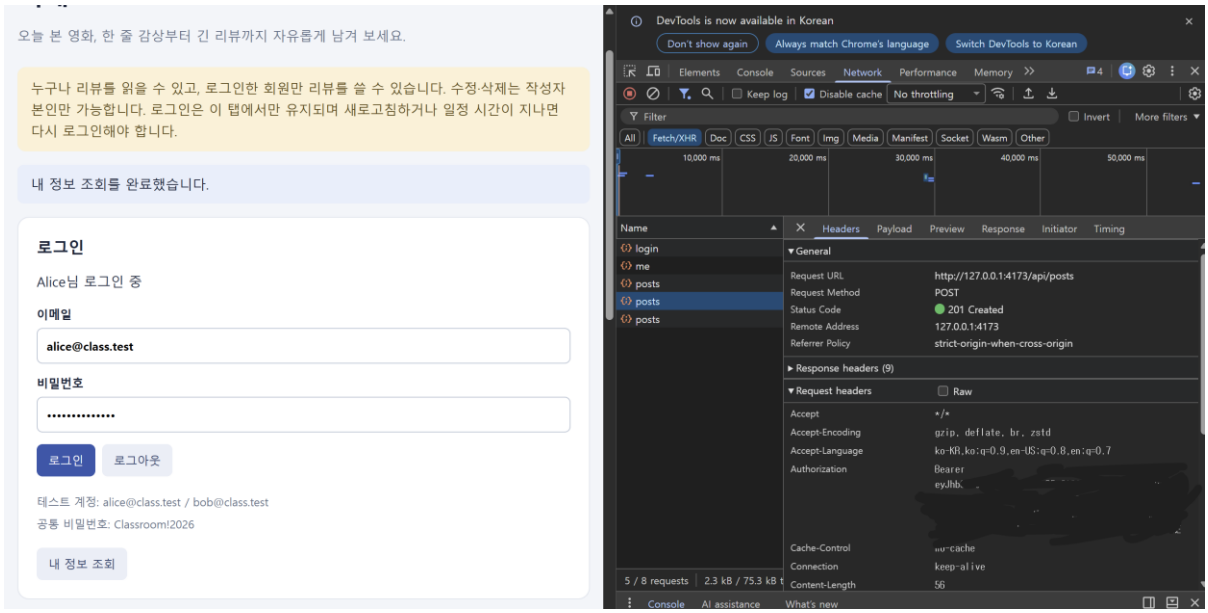
데이터 불변: T07·T08·T09·T10 에서 실패한 변경 요청 뒤 7 번 글을 다시 조회했을 때 title, content, updatedAt(2026-10-27T14:43:46.580Z)이 모두 그대로였다.

판단 순서: 토큰 없이 없는 글 수정 → 401 TOKEN_REQUIRED, 정상 토큰으로 없는 글 수정 → 404 NOT_FOUND, Bob 이 Alice 글에 공백 제목 → 403 FORBIDDEN, Alice 가 자기 글에 공백 제목 → 400 BAD_INPUT (order_check.png)

테스트 증거

로그인과 토큰 전송





다른 작성자의 수정과 삭제

```
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment> Tokens
>> Api GET /api/posts/7
>> Api PUT /api/posts/7 $bob '{"title":"hacked","content":"by bob"}'
>> Api DELETE /api/posts/7 $bob
>> Api GET /api/posts/7
토큰 새로 받음 (90초 유효)
200 {"id":7,"title":"타짜4","content":"포커를 잘친다","authorId":1,"authorName
":"Alice","createdAt":"2026-10-27T14:43:46.580Z","updatedAt":"2026-10-27T14:43
:46.580Z"}
403 {"code":"FORBIDDEN","message":"본인이 작성한 글만 수정·삭제할 수 있습니다.
"}
403 {"code":"FORBIDDEN","message":"본인이 작성한 글만 수정·삭제할 수 있습니다.
"}
200 {"id":7,"title":"타짜4","content":"포커를 잘친다","authorId":1,"authorName
":"Alice","createdAt":"2026-10-27T14:43:46.580Z","updatedAt":"2026-10-27T14:43
:46.580Z"}
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment>
```

```
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment> Tokens
>> Api POST /api/posts $bob '{"title":"fake","content":"x","authorId":1}'
>> Api POST /api/posts $bob '{"title":"fake","content":"x","userId":1}'
>> Api GET /api/posts
토큰 새로 받음 (90초 유효)
400 {"code":"BAD_INPUT","message":"허용된 JSON 필드만 보내세요."}
400 {"code":"BAD_INPUT","message":"허용된 JSON 필드만 보내세요."}
200 [{"id":7,"title":"타짜4","content":"포커를 잘친다","authorId":1,"authorNam
e":"Alice","createdAt":"2026-10-27T14:43:46.580Z","updatedAt":"2026-10-27T14:4
3:46.580Z"}]
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment>
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment>
```

```
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment> Tokens
>> Api PUT /api/posts/99999 $null '{"title":"t","content":"c"}'
>> Api PUT /api/posts/99999 $alice '{"title":"t","content":"c"}'
>> Api PUT /api/posts/7 $bob '{"title":"   ","content":"x"}'
>> Api PUT /api/posts/7 $alice '{"title":"   ","content":"x"}'
토큰 새로 받음 (90초 유효)
401 {"code":"TOKEN_REQUIRED","message":"로그인이 필요합니다."}
404 {"code":"NOT_FOUND","message":"게시글이 없습니다."}
403 {"code":"FORBIDDEN","message":"본인이 작성한 글만 수정·삭제할 수 있습니다.
"}
400 {"code":"BAD_INPUT","message":"title 입력을 확인하세요."}
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment>
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment>
```

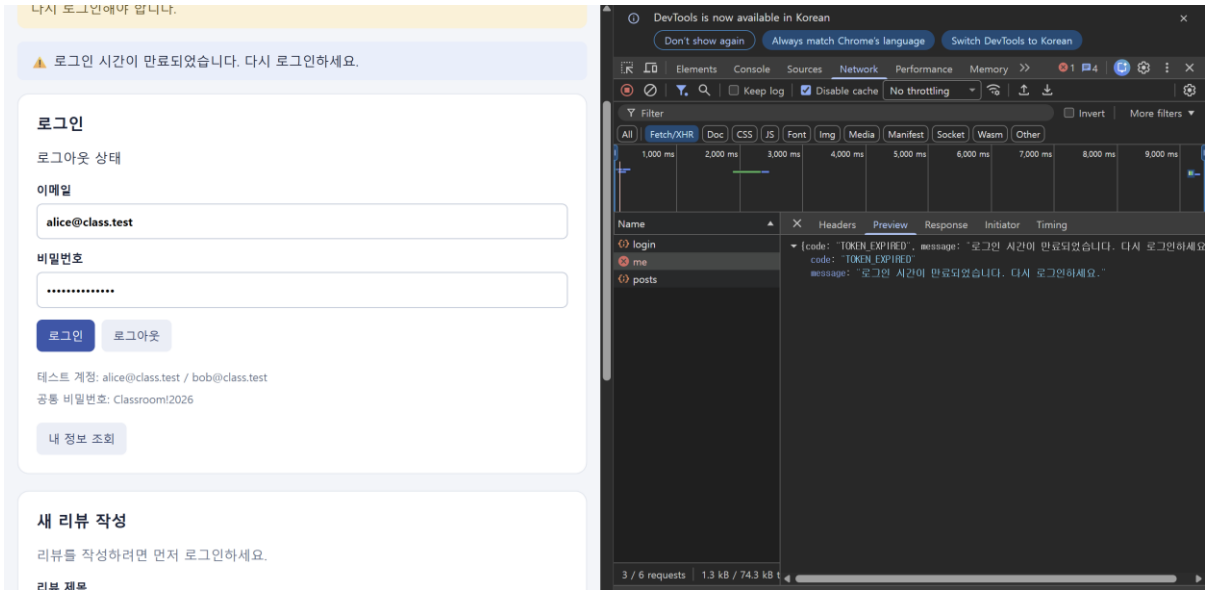
토큰 변조와 만료

```
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment> curl.exe -i http://127.0.
0.1:8000/api/me
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
WWW-Authenticate: Bearer
Content-Type: application/json; charset=utf-8
Content-Length: 67
ETag: W/"43-RGNN+FZWjmqoFuTETgzIb7JzL4Y"
Date: Tue, 27 Oct 2026 16:09:11 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"code":"TOKEN_REQUIRED","message":"로그인이 필요합니다."}
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment> Api POST /api/posts $null
 '{"title":"no token","content":"x"}'
401 {"code":"TOKEN_REQUIRED","message":"로그인이 필요합니다."}
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment>
```

```
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment> Tokens
>> $p = $alice.Split("."); $s = $p[2]
>> $p[2] = $(if ($s[0] -eq "A") { "B" } else { "A" }) + $s.Substring(1)
>> $bad = $p -join "."
>> curl.exe -i -H "Authorization: Bearer $bad" http://127.0.0.1:8000/api/me
>> Api POST /api/posts $bad '{"title":"tampered","content":"x"}'
>> Api PUT /api/posts/7 $bad '{"title":"tampered","content":"x"}'
>> Api DELETE /api/posts/7 $bad
>> Api GET /api/posts/7
토큰 새로 받음 (90초 유효)
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
WWW-Authenticate: Bearer
Content-Type: application/json; charset=utf-8
Content-Length: 73
ETag: W/"49-jFfARR+3GG3/bVq4C7TovdacoaM"
Date: Tue, 27 Oct 2026 16:34:14 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}401 {"code":"INV
ALID_TOKEN","message":"유효하지 않은 토큰입니다."}
401 {"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}
401 {"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}
200 {"id":7,"title":"타짜4","content":"포커를 잘친다","authorId":1,"authorName
":"Alice","createdAt":"2026-10-27T14:43:46.580Z","updatedAt":"2026-10-27T14:43
:46.580Z"}
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment>
```



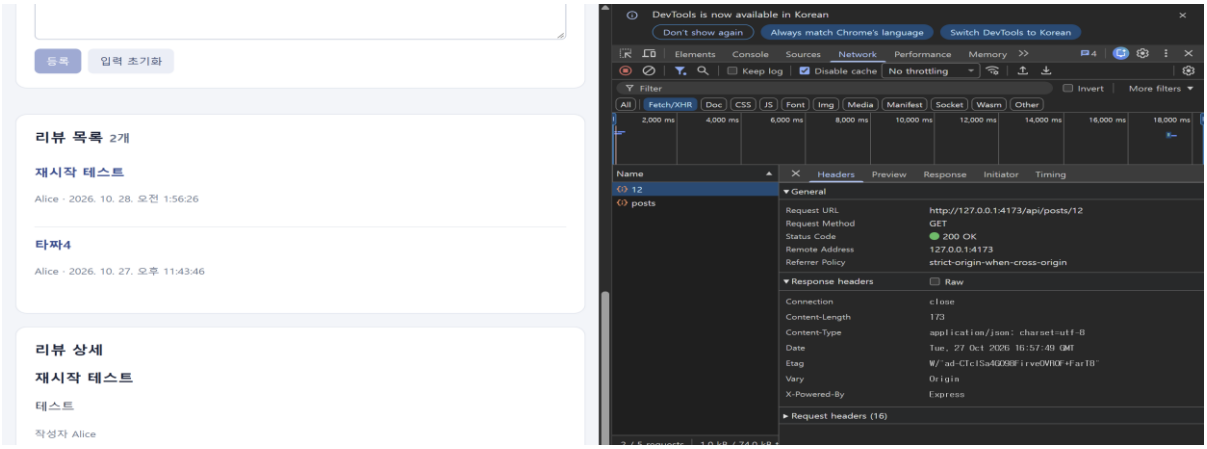

```
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment> $old = Login "alice@class
.test"
>> Start-Sleep -Seconds 5
>> Api PUT /api/posts/7 $old '{"title":"expired","content":"x"}'
>> Api DELETE /api/posts/7 $old
>> Api GET /api/posts/7
401 {"code":"TOKEN_EXPIRED","message":"로그인 시간이 만료되었습니다. 다시 로그
인하세요."}
401 {"code":"TOKEN_EXPIRED","message":"로그인 시간이 만료되었습니다. 다시 로그
인하세요."}
200 {"id":7,"title":"타짜4","content":"포커를 잘친다","authorId":1,"authorName
":"Alice","createdAt":"2026-10-27T14:43:46.580Z","updatedAt":"2026-10-27T14:43
:46.580Z"}
PS C:\Users\junho\OneDrive\바탕 화면\JWT-Assignment>
```

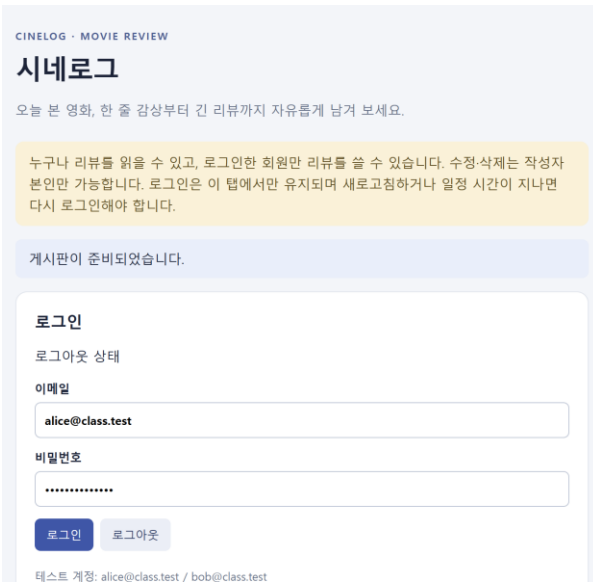
자동 검사와 로컬 배포 증거

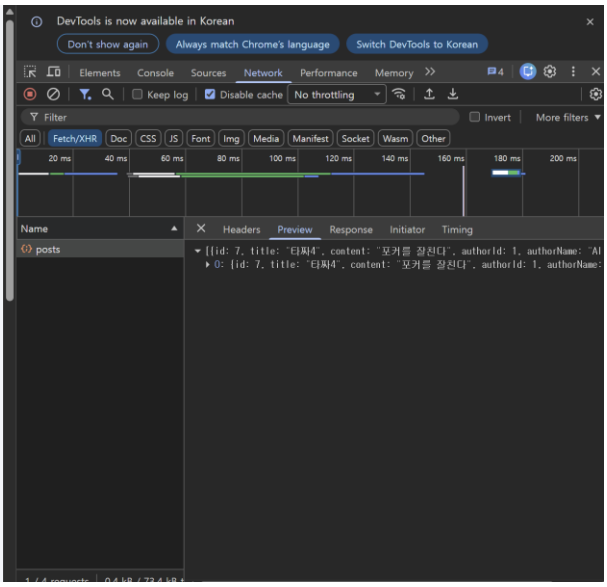
자동 검사: 새로 clone 한 폴더에서 백엔드를 실행한 상태로 py tools\check_api.py --mode final 을 실행했다. 모든 항목이 PASS 였다.

빌드와 4173 번 API 연결: 빌드 화면(4173)에서 보낸 로그인 요청이 http://127.0.0.1:4173/api/auth/login 으로 나가 200 을 받았다 (T02_login_status.png).

재시작 유지: 백엔드를 종료했다가 다시 실행한 뒤에도 "재시작 테스트" 글이 작성자·내용과 함께 남아 있었다. 새로고침하면 로그인은 풀리지만 글 목록은 유지된다







AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: 사용

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Claude (Anthropic), Claude Opus 5.5	학습·설계	과제 요구사항 정리, React+Express 선택	참고
Claude (Anthropic), Claude Opus 5.5	디버깅	PowerShell 실행 정책 오류, 304 캐시 응답, 저장 누락으로 인한 200 응답 원인 확인	참고
Claude (Anthropic), Claude Opus 5.5	코드 생성	auth.js (loadConfig, issueToken, currentUserId, assertOwner)	그대로 반영 후 실행 검증
Claude (Anthropic), Claude Opus 5.5	코드 생성	api.js (login, authorizationHeaders, request, canEdit), main.jsx (run, 배너, 로그아웃, 작성 안내)	그대로 반영 후 실행 검증
Claude (Anthropic), Claude Opus 5.5	테스트	PowerShell 도우미 함수(Login, Tokens, Api), T01~T17 명령	그대로 사용
Claude (Anthropic), Claude Opus 5.5	문장 정리	README.md, 이 보고서 초안	초안 받은 뒤 본인 수정

대표 사례 1 - 서버 JWT 검증 (currentUserId)

도구 / 날짜: Claude / 2026-10-27

요청한 내용: BE-02: currentUserId 의 데모 ID 반환을 없애고 토큰 없음·변조·만료를 구분하는 검증 구현

AI 가 제안한 내용: Bearer 정규식 추출 → jwt.verify(algorithms HS256, issuer, audience) → TokenExpiredError 를 TOKEN_EXPIRED 로 구분 → exp 존재·sub 형식·실제 계정 확인

실제 반영 위치: backend-express/src/auth.js currentUserId()

내가 확인하고 수정한 내용: 코드는 그대로 반영했다. 처음 시험에서 토큰 없이 /me 를 요청했는데 401 이 아니라 200 이 나왔다. 확인해 보니 auth.js 를 저장하지 않아 서버가 이전 코드(return 1)로 실행되고 있었다. 저장하고 서버를 재시작한 뒤 다시 시험했다. 라이브러리가 exp 없는 토큰을 통과시킨다는 설명을 듣고, exp 를 직접 확인하는 줄이 필요한 이유를 이해했다.

검증 방법과 결과: T04 401 TOKEN_REQUIRED, T09 서명 변조 401 INVALID_TOKEN, T10 TTL 3 초 만료 401 TOKEN_EXPIRED, 정상 토큰 /me 200

대표 사례 2 - 작성자 권한 검사 (assertOwner)와 서버 직접 요청 테스트

도구 / 날짜: Claude / 2026-10-27

요청한 내용 : BE-03 작성자 권한 구현과, 화면 버튼과 상관없이 서버 권한을 확인하는 방법

AI 가 제안한 내용 : 검증된 userId 와 글의 authorId 를 비교해 다르면 403 을 던지는 assertOwner(), 그리고 PowerShell 에서 Bob 토큰으로 직접 PUT·DELETE 를 보내는 도우미 함수

실제 반영 위치 : backend-express/src/auth.js assertOwner(), 테스트용 PowerShell 함수(저장소에는 미포함)

내가 확인하고 수정한 내용 : 코드는 그대로 반영했다. 테스트할 때 글 번호와 사용자 ID(Bob=2)를 헷갈려 2 번 글로 요청했다가 404 가 나왔고, 응답의 Location 헤더에서 실제 글 번호(11)를 확인해 다시 시험했다.

검증 방법과 결과 : T07·T08 403 FORBIDDEN 후 재조회 시 updatedAt 까지 동일, T17 Bob 본인 글 200·204, 판단 순서 401→404→403→400

대표 사례 3 - 프론트 토큰 전송과 응답 코드별 처리 (api.js)

도구 / 날짜: Claude / 2026-10-27

요청한 내용 : FE-01~04: 로그인 연결, Bearer 헤더 첨부, 401·403 처리, 작성자별 버튼

AI 가 제안한 내용 : 토큰을 메모리 변수에만 저장, 토큰이 있을 때만 헤더 생성, 401 이면 clearSession(), code 별 안내 문구, canEdit()로 작성자 비교

실제 반영한 위치 : frontend-react/src/api.js 전체, frontend-react/src/main.jsx run()

내가 확인하고 수정한 내용 : 코드는 그대로 반영하고 화면 문구를 영화 리뷰 주제에 맞게 바꿨다. Network 확인 중 목록 요청이 304 로 표시되어, 브라우저 캐시 때문이라는 설명을 듣고 Disable cache 를 켠 뒤 200 을 확인했다.

검증 방법과 결과 : T02 로그인 표시, T05 Authorization 헤더 첨부, T01 공개 조회 시 헤더 없음, T10·T11 401 시 자동 로그아웃, T16 새로고침 시 로그인 해제

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
수업 제공 시작 코드와 과제 문서(02_API 명세.md, 05_테스트시나리오.md 등)	전체	app.js·store.js·users.js·check_api.py 는 제공 코드 그대로 사용
jsonwebtoken 문서 https://github.com/auth0/node-jsonwebtoken	auth.js	jwt.sign / jwt.verify 옵션(algorithms, issuer, audience, expiresIn)

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

본문의 authorId 를 믿으면 Bob 이 authorId:1 을 넣어 Alice 이름으로 글을 쓸 수 있다. 내 코드는 두 단계로 막는다. 먼저 app.js 의 fields()가 title 과 content 가 아닌 필드가 오면 400 BAD_INPUT 으로 거부한다. 그리고 작성자는 본문이 아니라 currentUserId()가 검증한 토큰의 sub 로 서버가 정한다. T12 에서 authorId 와 userId 를 각각 넣었을 때 모두 400 이 나오고 위조 글이 생기지 않았다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼은 화면에만 있는 것이라 curl 이나 PowerShell 로 API 를 직접 호출하면 우회할 수 있다. 서버에서는 auth.js 의 assertOwner()가 검증된 사용자 ID 와 저장된 글의 authorId 를 비교해 다르면 403 을 반환한다. T07·T08 에서 버튼 없이 Bob 의 토큰으로 직접 요청해도 403 이 나왔고 글은 바뀌지 않았다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 토큰의 payload 를 Base64 로 풀어서 읽기만 하므로 서명이 위조돼도 그대로 읽힌다. verify 는 서버 키로 서명을 다시 계산해 비교하고 만료 등도 확인한다. 내 코드는 jwt.verify()로 HS256 알고리즘 고정, 서명, exp 만료, iss, aud 를 검증하고, 추가로 exp 가 있는지, sub 가 양의 정수 문자열인지, 실제 계정이 있는지를 확인한다. T09 에서 서명 한 글자만 바꿔도 401 INVALID_TOKEN 이 나왔다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 브라우저 메모리의 토큰과 사용자 정보, 화면의 로그인 표시를 지운다. 토큰 만료는 서버가 exp 를 보고 401 TOKEN_EXPIRED 를 반환하는 것이고, 프론트는 이 응답을 받으면 로그인 상태를 해제한다. 새로고침은 JavaScript 메모리가 초기화되어 로그인 상태가 사라지지만, 게시글은 SQLite 에 있어서 그대로 남는다(T16). 로그아웃은 서버에서 토큰을 무효화하지 않기 때문에, 이미 복사된 토큰은 exp(기본 90 초)가 지날 때까지 계속 쓸 수 있다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

BE-02 를 구현한 뒤 토큰 없이 /api/me 를 요청했는데 401 이 아니라 200 과 Alice 정보가 나왔다. 코드가 틀렸는지 의심했지만, 결과를 보면 서버가 아직 데모 ID 1 을 반환하는 시작 코드로 동작하고 있었다. VS Code 탭을 확인해 보니 auth.js 가 저장되지 않은 상태였다. 파일을 저장하고 백엔드를 Ctrl+C 로 종료했다가 npm start 로 다시 실행했다. 재검증에서 토큰 없음 401 TOKEN_REQUIRED, 정상 토큰 200, 서명 변조 401 INVALID_TOKEN 을 확인했다. 이후에는 코드 수정, 저장, 서버 재시작, 시험 순서를 지켰다. 그 밖에 Windows PowerShell 실행 정책 때문에 npm 이 실행되지 않은 문제는 Set-ExecutionPolicy -Scope CurrentUser RemoteSigned 로 해결했다.

현재 남은 문제와 다음에 개선할 점: 로그아웃 해도 이미 발급된 토큰은 만료 전까지 유효함.

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거

GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL 과 커밋 SHA 제출